Therefore, $L^O = \max(\ell_i^O, \ell_j^O) = \ell_j^O$. Since $L^G = \max(\ell_i^G, \ell_j^G)$, in order to show that $L^G \leq L^O$, it suffices to show that $\ell_i^G \leq L^O$ and $\ell_j^G \leq L^O$. By definition we have

$$\ell_i^G \;=\; t + (t_i + t_j) - d_i \;<\; t + (t_i + t_j) - d_j \;=\; \ell_j^O \;=\; L^O,$$

and

$$\ell_j^G \;=\; t + t_j - d_j \;\leq\; t + (t_i + t_j) - d_j \;=\; \ell_j^O \;=\; L^O.$$

Therefore, we have $L^G = \max(\ell_i^G, \ell_j^G) \leq L^O$, as desired. In conclusion, we have the following.

Claim: The greedy scheduling algorithm minimizes maximum lateness.

Lecture 11: Greedy Approximation Algorithms: The k-Center Problem

Greedy Approximation for NP-Hard Problems: As algorithm design methods go, the greedy approach is not very powerful. Nonetheless, one of the common applications of greedy algorithms is for producing approximation solutions to NP-hard problems.

As we shall see later this semester, NP-hard optimization problems represent very challenging computational problems in the sense that there is no known exact solution that has worst-case polynomial-time running time. Given an NP-hard problem, there are no ideal algorithmic solutions. One has to compromise between optimality or running time. Nonetheless, there are are number of examples of NP-hard problems where simple greedy heuristics produce solutions that are not far from optimal.

Clustering and Facility Location: Clustering is among the most widely studied problems in computer science. In a nutshell, clustering problems involve taking a very large set of objects, called *points*, and selecting a much smaller set of representative points. For example, companies like Netflix want to develop profiles of their users in order to recommend good programs and films to show them. Based on the available data about a user (demographic, geographic, language, and past preferences), they would like to classify each user as belonging to one (or generally a few) of a relatively small set of representative categories that will reflect the films that they like.

One way to approach this problem is to represent the information about each user as a long vector of numerical properties that encode this user's individual characteristics. Thus, each user is mapped to a point in a multi-dimensional space of *user characteristics*. It is possible to compute the distance between any pair of points within this space. The hope is that if two users have similar characteristics, that is, if the distance between their property vectors is small, then these users are expected to have similar interests.

Suppose that Netflix decides that it is going to partition its tens of millions of users into a small number k (perhaps tens to hundreds) of clusters. We will identify each cluster by specifying a central point of the cluster, its *cluster center*. To define the problem formally, we need some *objective function* by which to measure the quality of our solution. A simple

way is based on minimizing the distance of each point to its closest cluster center. This gives rise to the k-center problem, which we define next.

The k-Center Problem: Let us suppose that we are given a set P of n points in space. For each pair of points $u, v \in P$, let $\delta(u, v)$ denote the distance between u and v. We will assume that the distance satisfies the typical properties of any natural distance function:

Positivity: $\delta(u, v) \geq 0$ and $\delta(u, v) = 0$ if and only if $u = v$.

Symmetry: $\delta(u, v) = \delta(v, u)$

Triangle inequality: $\delta(u, w) \leq \delta(u, v) + \delta(v, w)$

These properties hold both for geometric distances, like the Euclidean distance, as well graph-based distances, like the shortest path between two nodes of a weighted graph.

We are also given a number k, which is presumably much much smaller than n. In the *k-center problem* the objective is to identify a subset C of k points of P, called *cluster centers*, so as to minimize the maximum distance of every point of P to its closest point in C.

A Geometric Perspective: Given a set $C = \{c_1, \ldots, c_k\}$ of cluster centers and $c_i \in C$ we define its *neighborhood*, denoted $N(c_i)$, to be the set of points u of P such that c_i is the closest center to u, that is

$$N(c_i) \;=\; \{v \in P \mid \delta(v, c_i) \leq \delta(v, c_j), \text{ for } i \neq j\}.$$

Observe that every point of $N(c_i)$ lies within distance $\Delta(C)$ of c_i. This suggests a more visual way of interpretting the k-center problem. In particular, if the distance is the Euclidean distance and P are points in the plane, then this means that every point of $N(c_i)$ lies within a circular disk of radius $\Delta(C)$ centered at c_i. Thus, the k-center problem is equivalent to determining the smallest radius r such that all the points of P are contained within k disks of radius r, where each disk is centered at some point of P. (The radius r is equal to $\Delta(C)$ and the centers of the disks are the cluster centers. See Fig. 41.)

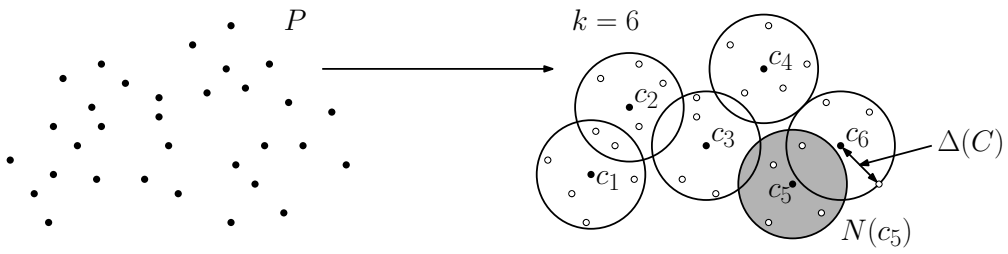


Fig. 41: The k-center problem in the Euclidean plane.

You might wonder why we constrain the cluster centers to be points of P. In fact there is no compelling reason. The version of k-center where centers must be chosen from P is sometimes referred to as the *discrete k-center problem*, and the greedy approach assumes this version of the problem. An more general form is one in which we have two sets P and Q, where the points of P are to be covered by the disks, and the cluster centers must be chosen from Q. (Thus, in our version $P = Q$.)

Let us assume for simplicity that there are no ties for the distances to the closest center (or that any such ties have been broken arbitrarily). Then $N(c_1), \ldots, N(c_k)$ defines a *partition* of P. The *bottleneck distance* associated with each center is defined to be the distance to its farthest point in $N(c_i)$, that is,

$$\Delta(c_i) = \max_{v \in N(c_i)} \delta(v, c_i).$$

Finally, we define the overall *bottleneck distance* to be

$$\Delta(C) = \max_{c_i \in C} \Delta(c_i).$$

This is the maximum distance of any vertex from its nearest center.

Intuitively, if we think of the cluster centers as the locations of Starbucks (or your favorite retailer), then each customer (point in P) is associated with the closest Starbucks. The set $N(c_i)$ are the customers that go to the ith Starbucks location. The value $\Delta(c_i)$ is the maximum distance any of these customers needs to travel, and $\Delta(C)$ is the maximum distance that *anyone* needs to travel to their nearest Starbucks.

Greedy Approximation Algorithm: Later in the semester we will show that the k-center problem is NP-hard. (This assumes that k is an arbitrary value between 1 and n, and hence it can grow asymptotically with n. When k is a fixed constant, the problem can be solved in $O(n^{k+1})$ time, which is a polynomial.) We will consider a simple approximation algorithm for this problem. Given P and k, this algorithm produces a set of centers such that the final bottleneck distance is at most twice the optimum value.

The algorithm begins by selecting any point of P to be the initial center g_1. We then repeat the following process until we have k centers. Let $G_i = \{g_1, \ldots, g_i\}$ denote the current set of centers. Recall that $\Delta(G_i)$ is the maximum distance of any point of P from its nearest center. Let u be the point achieving this distance. The greediest way to satisfy u is to put a center directly at u, that is, set $g_{i+1} \leftarrow u$. In other words, set $G_{i+1} \leftarrow G_i \cup \{u\}$.

Following our Starbucks analogy, the point u selected in each step is the most dissatisfied customer, because she has to drive the farthest to get to the nearest Starbucks. However, she is rewarded by having a new Starbucks plopped down on top of her house.

The pseudocode is presented in the code block below. The value $d[u]$ denotes the distance from u to its closest center. (We make one simplification, by starting with G being empty. When we select the first center, all the points of P have infinite distances, so the initial choice is arbitrary.)

It is easy to see that the algorithm's running time is $O(kn)$. In general $k \leq n$, so this is $O(n^2)$ in the worst case. For the sake of the analysis, define $\Delta_i = \Delta(G_i)$. The algorithm is illustrated (for the Euclidean case) in Fig. 42. In each step we show the current set of centers $G_i = \{g_1, \ldots, g_i\}$ and the value Δ_i, which is the farthest from its closest center. (Note that the closest center can be any point of G_i.) The point that achieves this distance is labeled as g_{i+1}, since it will be the next center chosen.

Approximation Bound: Let $G = \{g_1, \ldots, g_k\}$ denote some set of centers computed by the greedy algorithm, and let $\Delta(G)$ denote its bottleneck distance. Let $O = \{o_1, \ldots, o_k\}$ denote any

```
GreedyKCenter(P, k) {
    G = empty
    for each u in P do                // initialize distances
        d[u] = INFINITY

    for (i = 1 to k) {
        Let u be the point of P such that d[u] is maximum
        Add u to G                    // u is the next cluster center
        for (each v in P) {           // update distance to nearest center
            d[v] = min(d[v], delta(v,u))
    }
    return G                          // final centers
}
```

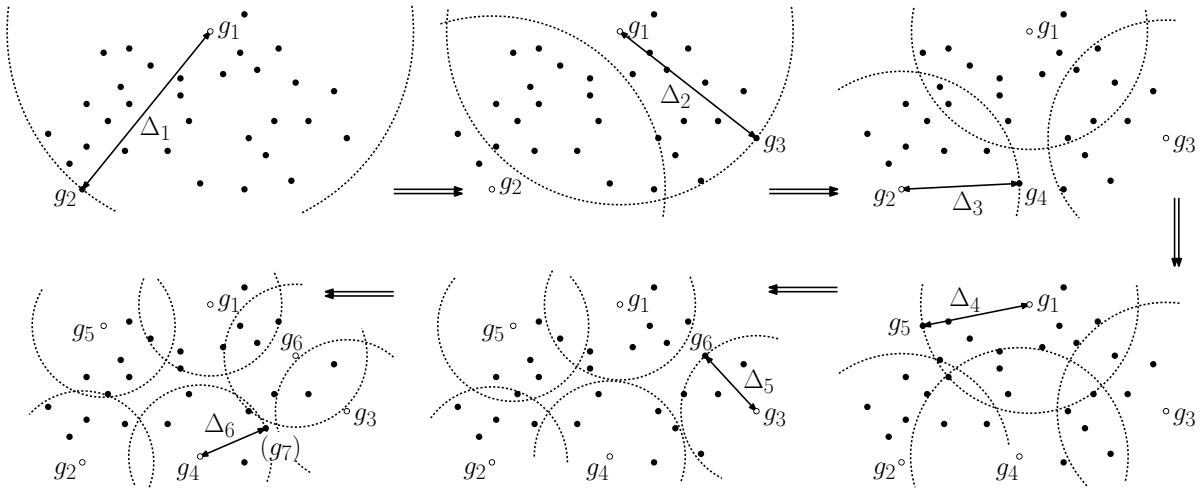


Fig. 42: Greedy approximation to k-center (for $k = 6$).

optimum set of centers, and let $\Delta(O)$ denote the optimum bottleneck distance. Our main result is

$$\Delta(G) \;\leq\; 2\Delta(O),$$

which implies that the greedy approximation algorithm is within a factor of 2 of the optimum.

Our approach will be similar to one that we have used in other greedy proofs. We will present a lower bound on the optimum bottleneck distance, and we will show that our algorithm produces a value that is at most twice this optimum value. The analysis is based on the following three lemmas, each of which is quite straightforward to prove. The greedy algorithm stops with g_k, but for the sake of the analysis it is convenient to consider the next center to be added if we ran it for one more iteration. That is, define g_{k+1} to be the point of P that is maximizes the distance to its closest center in G_k. (This distance is $\Delta(G_k)$.) Also, define $G_{k+1} = \{g_1, \ldots, g_{k+1}\}$.

Lemma 1: The sequence of bottleneck distances $\langle \Delta_1, \ldots, \Delta_k \rangle$ in the greedy algorithm is monotonically nonincreasing, that is $\Delta_i \geq \Delta_{i+1}$. (In Fig. 42 this is represented by the fact that the radii of the disks decreases with each stage.)

Lemma 2: Every pair of greedy centers is separated by at least the greedy bottleneck distance, that is, for $1 \leq i < j \leq k+1$, $\delta(g_i, g_j) \geq \Delta(G)$. (In Fig. 42 this is represented by the fact that no disk in the final picture contains the center of any other.)

Lemma 3: Let $\Delta_{\min} = \Delta(G)/2$. Then for any set C of k cluster centers, we have $\Delta(C) \geq \Delta_{\min}$. (In Fig. 42 this means that it is *not* possible to cover all the points of P using k disks whose radii are smaller than $\Delta_{\min}$.)

Before getting to the proofs, let's see why this implies the approximation bound. By Lemma 3, the bottleneck cost of any k-center solution C must be at least as large as $\Delta_{\min} = \Delta(G)/2$. Thus, $\Delta_{\min}$ plays the role of the lower bound, which we mentioned earlier. Since this applies to the optimal solution as well, we have $\Delta(O) \geq \Delta(G)/2$, or equivalently $\Delta(G) \leq 2\Delta(O)$. Thus, G is a factor-2 approximation to the optimal bottleneck distance.

Here are the proofs of the lemmas:

Proof of Lemma 1: As more centers are added, the distance of any point to its closest center cannot increase (it can only stay the same or decrease). Therefore, since $G_i \subseteq G_{i+1}$, we have $\Delta_i \geq \Delta_{i+1}$.

Proof of Lemma 2: When g_j is added, it is at distance Δ_{i-1} from its closest center in G_{j-1}. Since $i < j$, $g_i \in G_{j-1}$, and therefore $\delta(g_i, g_j) \geq \Delta_{i-1}$, and by Lemma 1, this is at least as large as $\Delta_k = \Delta(G)$.

Proof of Lemma 3: Consider any set C of k centers whose bottleneck distance is smaller than $\Delta_{\min}$. By the pigeonhole principal, there exists at least two centers $g_i, g_j \in G_{k+1}$, where $1 \leq i < j \leq k+1$ that are in the same neighborhood of some center $c \in C$. Since $\Delta(c) \leq \Delta_{\min}$, by combining Lemma 2 with the triangle inequality we have

$$\Delta(G) \;\leq\; \delta(g_i, g_j) \;\leq\; \delta(g_i, c) + \delta(c, g_j) \;<\; \Delta(C) + \Delta(C) \;=\; 2\Delta(C).$$

Therefore, $\Delta(C) > \Delta(G)/2 = \Delta_{\min}$. Since this holds for any set of centers, it holds for the optimum set O.

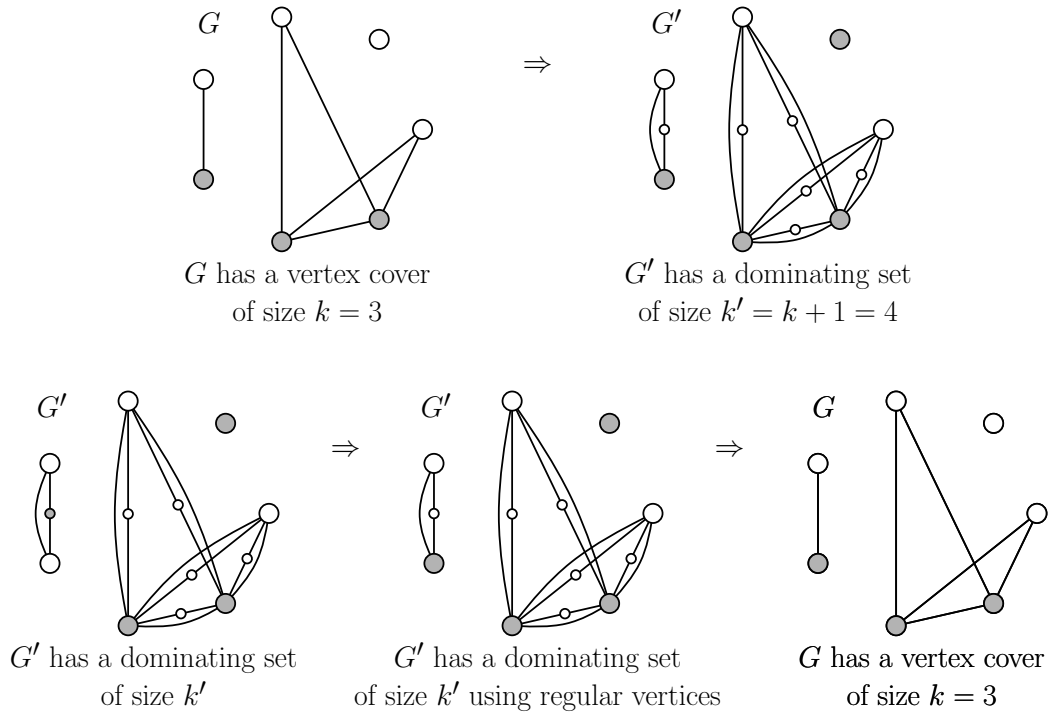


Fig. 88: Correctness of the VC to DS reduction (where $k = 3$ and $I = 1$).

We claim that V' is a vertex cover for G. If, to the contrary there were an edge $\{u, v\}$ of G that was not covered (neither u nor v was in V') then the special vertex w_{uv} would not be adjacent to any vertex of V'' in G', contradicting the hypothesis that V'' was a dominating set for G'.

Lecture 25: Approximation Algorithms: Vertex Cover and TSP

Coping with NP-completeness: With NP-completeness we have seen that there are many important optimization problems that are likely to be quite hard to solve exactly. Since these are important problems, we cannot simply give up at this point, since people do need solutions to these problems. How do we cope with NP-completeness:

Brute-force search: This is usually only a viable option for small input sizes (e.g., $n \leq 20$).

Heuristics: This is a strategy for producing a valid solution, but may be there no guarantee on how close it is to optimal.

General Search Algorithms: There are a number of very powerful techniques for solving general combinatorial optimization problems. These go under various names such as *branch-and-bound*, *Metropolis-Hastings*, *simulated annealing*, and *genetic algorithms*. The performance of these approaches varies considerably from one problem to problem and instance to instance. But in some cases they can perform quite well.

Approximation Algorithms: This is an algorithm that runs in polynomial time (ideally), and produces a solution that is guaranteed to be within some factor of the optimum solution.

Performance Bounds: Most NP-complete problems have been stated as decision problems for theoretical reasons. However underlying most of these problems is a natural optimization problem. For example, vertex cover optimization problem is to find the vertex cover of minimum size, the clique optimization problem is to find the clique of maximum size. An approximation algorithm is one that returns a legitimate answer, but not necessarily one of the optimal size.

How do we measure how good an approximation algorithm is? We define the *performance ratio* of an approximation algorithm as follows. Given an instance I of our problem, let $C(I)$ be the cost of the solution produced by our approximation algorithm, and let $C^*(I)$ be the optimal solution. We will assume that costs are strictly positive values. For a minimization problem we have $C(I)/C^*(I) \geq 1$. For a maximization problem we have $C^*(I)/C(I) \geq 1$. In either case, we want the ratio to be as small as possible. For any input size n, we say that the approximation algorithm achieves *performance ratio bound* $\rho(n)$, if for all I, $|I| = n$ we have

$$\max_I \left(\frac{C(I)}{C^*(I)}, \frac{C^*(I)}{C(I)} \right) \ \leq \ \rho(n).$$

Observe that $\rho(n)$ is equal to 1 if and only if the approximate solution is the true optimum solution.

Although NP-complete problems are equivalent with respect to whether they can be solved exactly in polynomial time in the worst case, their approximability varies considerably.

- Some NP-complete are *inapproximable* in the sense no polynomial time algorithm achieves a ratio bound smaller than ∞ unless P = NP.
- Some NP-complete can be approximated, but the ratio bound is a *function of* n. For example, the set cover problem (a generalization of the vertex cover problem), can be approximated to within a factor of $O(\log n)$.
- Some NP-complete problems can be approximated and the ratio bound is a *constant*.
- Some NP-complete problems can be approximated *arbitrarily well*. In particular, the user provides a parameter $\varepsilon > 0$ and the algorithm achieves a ratio bound of $(1 + \varepsilon)$. Of course, as ε approaches 0 the algorithm's running time gets worse. If such an algorithm runs in polynomial time for any fixed ε, it is called a *polynomial time approximation scheme* (PTAS).

Vertex Cover: We begin by showing that there is an approximation algorithm for vertex cover with a ratio bound of 2, that is, this algorithm will be guaranteed to find a vertex cover whose size is at most twice that of the optimum. Recall that a vertex cover is a subset of vertices such that every edge in the graph is incident to at least one of these vertices. The *vertex cover optimization problem* is to find a vertex cover of minimum size (See Fig. 89).

How does one go about finding an approximation algorithm. The first approach is to try something that seems like a "reasonably" good strategy, a *heuristic*. It turns out that many simple heuristics, when not optimal, can often be proved to be close to optimal.

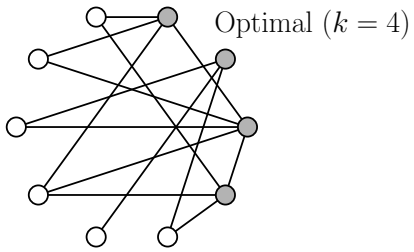


Fig. 89: Vertex cover (optimal solution).

Here is an very simple algorithm, that guarantees an approximation within a factor of 2 for the vertex cover problem. It is based on the following observation. Consider an arbitrary edge (u, v) in the graph. One of its two vertices *must* be in the cover, but we do not know which one. The idea of this heuristic is to simply put *both* vertices into the vertex cover. (You cannot get much stupider than this!)

We call this the *2-for-1 heuristic*. More formally, the algorithm runs in a series of stages. Initially the cover is empty. During each stage we select an arbitrary edge (u, v) from the graph and add both u and v to the current cover. We then remove *all* the edges that are incident to either u or v (since these edges are now all covered). We repeat until G has no more edges. (The algorithm shown in the following code fragment, and it is illustrated in Fig. 90.)

2-for-1 Approximation for VC

```
    two-for-one-VC(G=(V,E)) {
        C = empty
        while (E is nonempty) do {
(*)         let (u,v) be any edge of E
            add both u and v to C
            remove from E all edges incident to either u or v
        }
        return C
    }
```

Claim: The 2-for-1 approximation for VC achieves a performance ratio of 2.

Proof: returns a vertex cover for G that is at most twice the size of the optimal vertex cover. Consider the set C output by two-for-one-VC(G). Let C^* be the optimum vertex cover. Let A be the set of edges selected by the line marked with "$(*)$" in the code fragment. Because we add both endpoints of each edge of A to C, we have $|C| = 2|A|$. However, the optimum vertex cover C^* must contain at least one of these two vertices. Therefore, we have $|C^*| \geq |A|$. Therefore

$$|C| \;=\; 2|A| \;\leq\; 2|C^*| \qquad \Rightarrow \qquad \frac{|C|}{|C^*|} \;\leq\; 2$$

as desired.

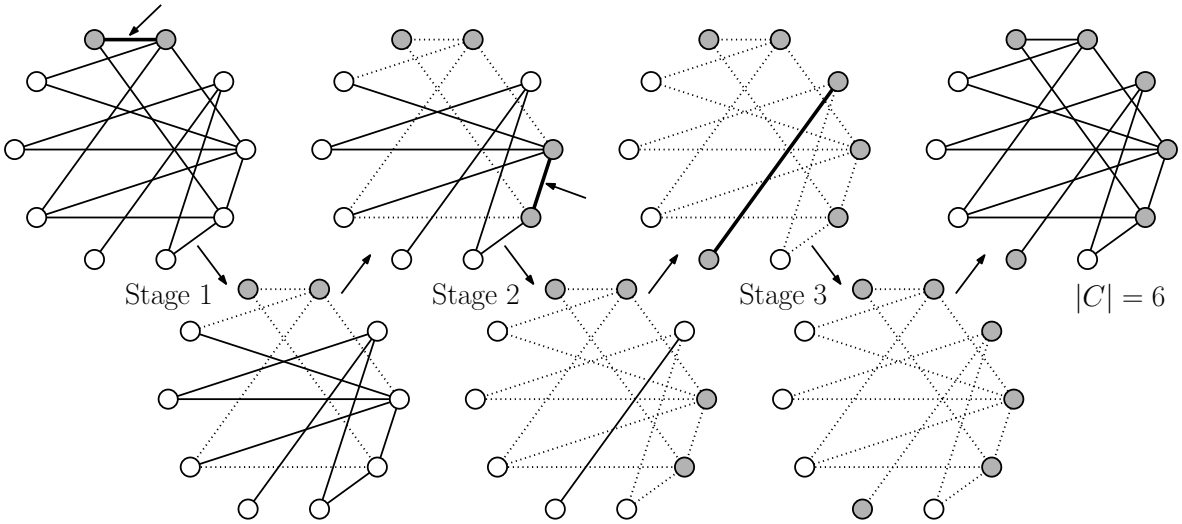


Fig. 90: The 2-for-1 heuristic for vertex cover.

This proof illustrates one of the main features of the analysis of any approximation algorithm. Namely, that we need some way of finding a bound on the optimal solution. (For minimization problems we want a lower bound, for maximization problems an upper bound.) The bound should be related to something that we can compute in polynomial time. In this case, the bound is related to the set of edges A, which form a maximal independent set of edges.

The Greedy Heuristic: It seems that there is a very simple way to improve the 2-for-1 heuristic. This algorithm simply selects any edge, and adds both vertices to the cover. Instead, why not concentrate instead on vertices of high degree, since a vertex of high degree covers the maximum number of edges. This is greedy strategy. We saw in the minimum spanning tree and shortest path problems that greedy strategies were optimal.

Here is the greedy heuristic. Select the vertex with the maximum degree. Put this vertex in the cover. Then delete all the edges that are incident to this vertex (since they have been covered). Repeat the algorithm on the remaining graph, until no more edges remain. (This algorithm is given in the code fragment below and is illustrated in Fig. 91.)

Greedy Approximation for VC

```
greedy-VC(G=(V,E)) {
    C = empty
    while (E is nonempty) do {
        let u be the vertex of maximum degree in G
        add u to C
        remove from E all edges incident to u
    }
    return C
}
```

It is interesting to note that on the example shown in Fig. 91, the greedy heuristic actually

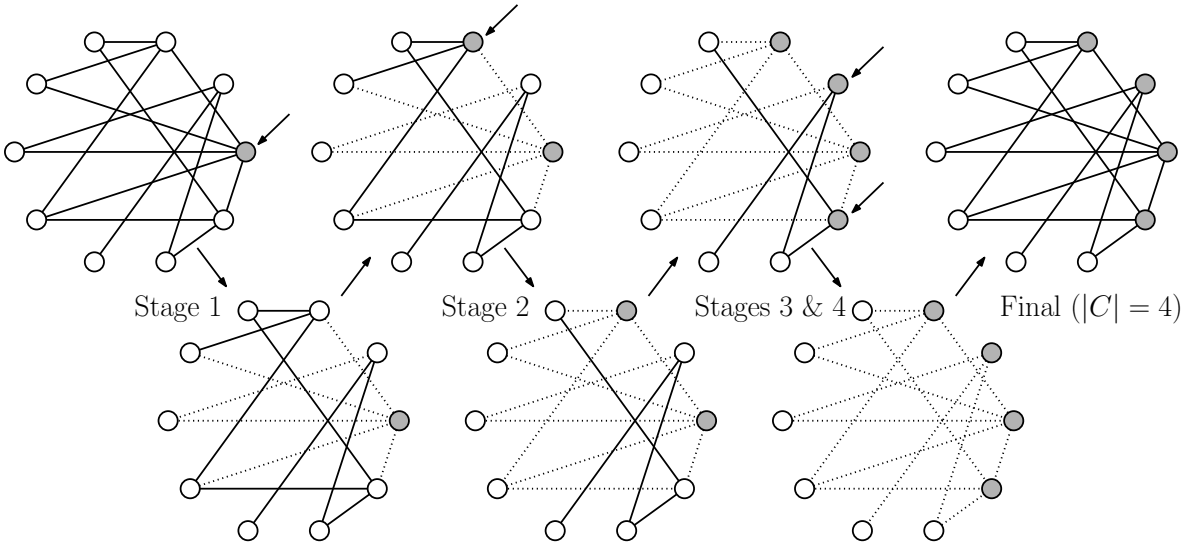


Fig. 91: The greedy heuristic for vertex cover.

succeeds in finding the optimum vertex cover. Given that it is more clever than the 2-for-1 heuristic, we might be inclined to conjecture that the greedy heuristic always does at least as well as the 2-for-1 heuristic. It is surprising, however, that answer is "no". Moreover, it can be shown that the greedy heuristic does *not* even achieve a constant performance bound. Indeed there exist graphs having n vertices such that the greedy heuristic achieves a performance ration of $\Theta(\log n)$. It should be mentioned, however, that experimental studies show that greedy actually works quite well in practice, and for "typical" graphs, it will perform better than the 2-for-1 heuristic.

Reductions and Approximations: Now that we have a factor-2 approximation for one NP-complete problem (vertex cover), you might be tempted to believe that we now have a factor-2 approximation for all NP-complete problems. Unfortunately, this is not true. The reason is that approximation factors are not generally preserved by transformations.

For example, recall that if V' is a vertex cover for G, then the complement vertex set, $V \setminus V'$, is an independent set for G. Suppose that G has n vertices, and a minimum vertex cover V' of size k. Then our heuristic is guaranteed to produce a vertex cover V'' that is of size at most $2k$. If we consider the complement set $V \setminus V'$, we know that G has a maximum independent set of size $n - k$. By complementing our approximation $V \setminus V''$ we have an "approximate" independent set of size $n - 2k$. How good is this? We achieve a performance ratio of

$$\rho(n, k) \;=\; \frac{n-k}{n-2k}.$$

The problem is that this ratio may be arbitrarily large. For example, if $n = 1001$ and $k = 500$, then the ratio is $501/(1001 - 1000) = 500/1 = 500$. This is terrible!

In summary, just because you can approximate one NP-complete problem, it does not follow that you approximate another.

Traveling Salesman with Triangle Inequality: In the Traveling Salesperson Problem (TSP) we are given a complete undirected graph with nonnegative edge weights, and we want to find a cycle that visits all vertices and is of minimum cost. Let $w(u, v)$ denote the weight on edge (u, v). Given a set of edges A forming a tour we define $W(A)$ to be the sum of edge weights in A.

For many of the applications of TSP, the edge weights satisfy a property called the *triangle inequality*. Intuitively, this says that the direct path from u to x, is never longer than an indirect path. More formally, for all $u, v, x \in V$

$$w(u, v) \leq w(u, x) + w(x, v).$$

There are many examples of graphs that satisfy the triangle inequality. For example, given any weighted graph, if we define $w(u, v)$ to be the shortest path length between u and v, then it will satisfy the triangle inequality. Another example is if we are given a set of points in the plane, and define a complete graph on these points, where $w(u, v)$ is defined to be the Euclidean distance between these points, then the triangle inequality is also satisfied.

When the underlying cost function satisfies the triangle inequality there is an approximation algorithm for TSP with a ratio-bound of 2. (In fact, there is a slightly more complex version of this algorithm that has a ratio bound of 1.5, but we will not discuss it.) Thus, although this algorithm does not produce an optimal tour, the tour that it produces cannot be worse than twice the cost of the optimal tour.

The key insight is to observe that a TSP with one edge removed is just a spanning tree. However it is not necessarily a minimum spanning tree. Therefore, the cost of the minimum TSP tour is at least as large as the cost of the MST. We can compute MST's efficiently, using, for example, Kruskal's algorithm. If we can find some way to convert the MST into a TSP tour while increasing its cost by at most a constant factor, then we will have an approximation for TSP. We shall see that if the edge weights satisfy the triangle inequality, then this is possible.

Here is how the algorithm works. Given any free tree there is a tour of the tree called a *twice around tour* that traverses the edges of the tree twice, once in each direction (see Fig. 92).

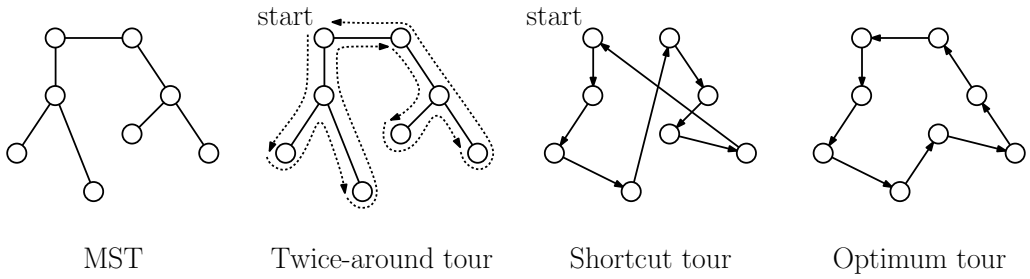


Fig. 92: TSP Approximation.

This path is not simple because it revisits vertices, but we can make it simple by *short-cutting*, that is, we skip over previously visited vertices. Notice that the final order in which vertices are visited using the short-cuts is exactly the same as a preorder traversal of the MST. (In fact, any subsequence of the twice-around tour which visits each vertex exactly once will

suffice.) The triangle inequality assures us that the path length will not increase when we take short-cuts.

TSP Approximation

```
approx-TSP(G=(V,E)) {
    T = minimum spanning tree for G
    r = any vertex
    H = list of vertices visited by a preorder walk of T starting at r
    return L
}
```

Claim: Approx-TSP achieves a performance ratio of 2.

Proof: Let H denote the tour produced by this algorithm and let H^* be the optimum tour. Let T be the minimum spanning tree. As we said before, since we can remove any edge of H^* resulting in a spanning tree, and since T is the minimum cost spanning tree we have

$$W(T) \ \leq \ W(H^*).$$

Now observe that the twice around tour of T has cost $2 \cdot W(T)$, since every edge in T is hit twice. By the triangle inequality, when we short-cut an edge of T to form H we do not increase the cost of the tour, and so we have

$$W(H) \ \leq \ 2 \cdot W(T).$$

Combining these we have

$$W(H) \ \leq \ 2 \cdot W(T) \ \leq \ 2 \cdot W(H^*) \quad \Rightarrow \quad \frac{W(H)}{W(H^*)} \leq 2,$$

as desired.

Lecture 37: Approximations: Bin Packing

Bin Packing: Bin packing is another well-known NP-complete problem. This is a partitioning problem where we are given a set of objects that are to be partitioned among a collection of containers, called *bins*. Each bin has the same capacity, and the objective is to use the smallest number of bins to hold all the objects.

More formally, we are given a set of n objects, where s_i denotes the *size* of the ith object. It will simplify the presentation to assume that the sizes have been normalized so that $0 < s_i < 1$. We want to put these objects into a set of bins. Each bin can hold a subset of objects whose total size is at most 1. The problem is to partition the objects among the bins so as to use the fewest possible bins. (Note that if your bin size is not 1, then you can reduce the problem into this form by simply dividing all sizes by the size of the bin.)

Bin packing arises in many applications. Many of these applications involve not only the size of the object but their geometric shape as well. For example, these include packing boxes into a truck, or cutting the maximum number of pieces of certain shapes out of a piece of sheet metal. However, even if we ignore the geometry, and just consider the sizes of the objects, the decision problem is still NP-complete. (The reduction is from the knapsack problem.)

Here is a simple heuristic algorithm for the bin packing problem, called the *first-fit heuristic*. We start with an unlimited number of empty bins. We take each object in turn, and find the first bin that has space to hold this object. We put this object in this bin. The algorithm is illustrated in Fig. 124. We claim that first-fit uses at most twice as many bins as the optimum. That is, if the optimal solution uses b_{opt} bins, and first-fit uses b_{ff} bins, then we show below that

$$\frac{b_{\text{ff}}}{b_{\text{opt}}} \leq 2.$$

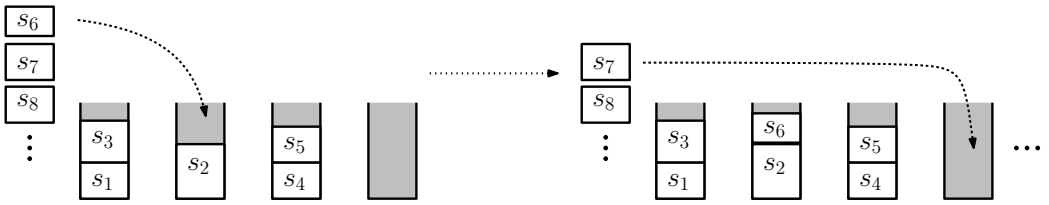


Fig. 124: First-fit Heuristic.

Theorem: The first-fit heuristic achieves a ratio bound of 2.

Proof: Consider an instance $\{s_1, \ldots, s_n\}$ of the bin packing problem. Let $S = \sum_i s_i$ denote the sum of all the object sizes. Let b_{opt} denote the optimal number of bins, and b_{ff} denote the number of bins used by first-fit.

First, observe that since no bin can hold more than one unit's worth of items, and we have a total of S units to be stored, it follows that we need a minimum of S bins to store everything. (And this would be achieved only if every bin were filled exactly to the top.) Thus, $b_{\text{opt}} \geq S$.

Next, we claim that $b_{\text{ff}} \leq 2S$. To see this, let t_i denote the total size of the objects that first-fit puts into bin i. There cannot be two bins $i < j$ such that $t_i + t_j < 1$. The reason

is that any item we decided to put into bin j must be small enough to fit into bin i. Thus, the first-fit algorithm would never put such an item into bin j. In particular, this implies that for all i, $t_i + t_{i+1} \geq 1$ (where indices are taken circularly modulo the number of bins). Thus we have

$$b_{\text{ff}} \;=\; \sum_{i=1}^{b_{\text{ff}}} 1 \;\leq\; \sum_{i=1}^{b_{\text{ff}}} (t_i + t_{i+1}) \;=\; \sum_{i=1}^{b_{\text{ff}}} t_i + \sum_{i=1}^{b_{\text{ff}}} t_{i+1} \;=\; S + S \;=\; 2S \;\leq\; 2b_{\text{opt}},$$

which completes the proof.

There are in fact a number of other heuristics for bin packing. Another example is *best-fit*, which attempts to put the object into the bin in which it fits most closely with the available space (assuming that there is sufficient available space). This is not necessarily a good idea, since it might tend to create very small spaces that will be hard to fill. There is also a variant of first-fit, called *first-fit-decreasing*, in which the objects are first sorted in decreasing order of size. (This makes intuitive sense, because it is best to first load the big items, and then try to squeeze the smaller objects into the remaining space.)

A more careful (an complicated) proof establishes that first-fit has a approximation ratio that is a bit smaller than 2, and in fact $17/10 = 1.7$ is possible. Best-fit has a very similar bound. It can be shown that first-fit-decreasing has a significantly better bound than either of these. In particular, it achieves a ratio bound of $11/9 \approx 1.222$.